Microprogrammed Control Unit Design

Week 7 — Storing the Control Sequence in Memory

Dr. Auqib Hamid Lone
Assistant Professor in Computer Science & Engineering

PCC CS-401: Computer Organization and Architecture

Department of Computer Science & Engineering
Government College of Engineering & Technology (GCET), Ganderbal

August 6, 2026

Where We Left Off (Week 6)

Last week: control signals generated by gates

- A **sequence counter (SC)** plus step decoder generated $T_1, T_2, \ldots$ automatically; an **opcode decoder** produced one-hot D_i lines.
- Every control signal was an **OR**, across instructions, of $(D_i \cdot T_j)$ terms — realized as actual AND/OR gates, wired by hand.

The gap we left open

Adding one instruction meant re-deriving Boolean equations and physically rewiring gates for every signal it touches. At ISA scale (dozens to hundreds of instructions) that design and verification cost explodes [3]. This week: an alternative that does not wire signals into gates at all.

Roadmap: Three Questions, One Thread

1. **Microinstructions and control memory** — what does it mean to *store* a control step instead of wiring it?
2. **Address sequencing** — if control words live in memory, what decides which one comes next?
3. **Horizontal vs. vertical encoding** — how wide does a stored control word need to be, and what does narrowing it cost?

Running thread for today

We rebuild the *same* R1 ← R2+R3 control unit from Weeks 5–6 — this time as a microprogram — so the hardwired-vs.- microprogrammed comparison is concrete, not just adjectives.

Today: what if we simply wrote the control signals down?

A Question We Skipped

Socratic question

Week 6's state table (Step → RTL → active signals) was already a complete specification of the control unit's behavior. We then spent two slides turning it into AND/OR gates. *Did we have to?*

A table is exactly the kind of thing hardware is good at storing and reading back — that is what memory does. **Microprogrammed control** stores the state table directly, as data, and reads one row per cycle instead of evaluating gates. The idea dates to Maurice Wilkes (1951), well before ISAs got large enough to need it.

Definition: Control Word / Microinstruction

Control word (microinstruction)

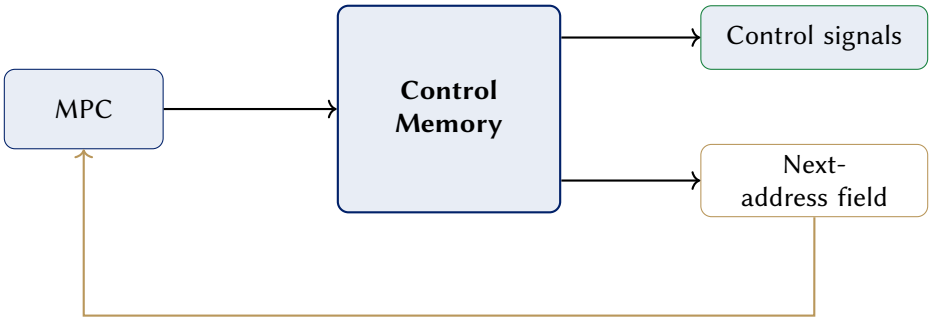
A **control word** is one row of stored control information: which control signals to assert *this* cycle, plus enough information to determine which control word to read *next*. A **microprogram** is the ordered sequence of control words that implements one machine instruction.

- Compare to Week 6: one row of the state table $\rightarrow$ one control word. Nothing about *what* is specified changes — only whether it is wired into gates or written into memory.
- New notation: **microaddress** — the address of a control word inside this memory (added to the course registry).

Control Memory (CM) and the Microprogram Counter (MPC)

Two new components

- **Control memory (CM)** — a small ROM/RAM whose rows are control words, indexed by microaddress.
- **Microprogram counter (MPC)** — a register holding the microaddress of the control word being read *this* cycle; the microprogrammed analogue of SC.



Worked Example: The Microprogram for R1 ← R2+R3

Microaddress	RTL	Control word (signals asserted)	Next
0	$Y \leftarrow [R2]$	$R2_{\text{out}}$, Y_{in}	1
1	$Z \leftarrow [Y] + [R3]$	$R3_{\text{out}}$, ALU=add, Z_{in}	2
2	$R1 \leftarrow [Z]$	Z_{out}, $R1_{\text{in}}$, END	fetch

Line up against Week 6

Same three rows, same signals, same order as the hardwired state table — we have only changed *where* they live: three rows of CM instead of three rows of a truth table destined for gates.

But CM has three rows here — how does hardware know which row is next, in general?

The Missing Piece: Where Does MPC Start?

Socratic question

SC in Week 6 always started an instruction at T_1 (address 0 in our new language). But CM stores the microprograms for *every* instruction back to back — ADD's three rows, SUB's three rows, LOAD's rows, and so on. If MPC always started at 0, it would only ever run ADD.

Something must translate "the opcode in IR is ADD" into "load MPC with *this instruction's* starting microaddress." That translation is **address sequencing**.

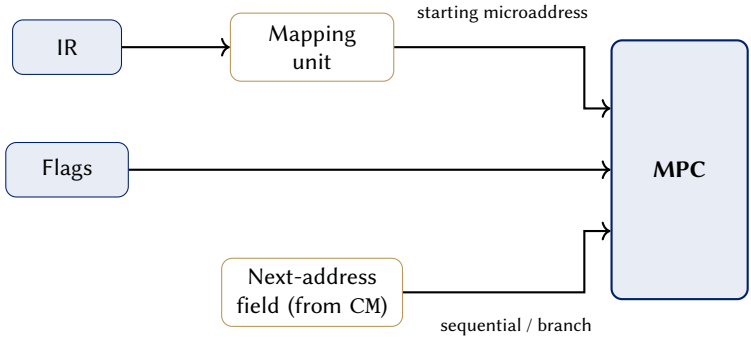
Mapping: Opcode → Starting Microaddress

Mapping logic

On every instruction fetch, a small **mapping** unit converts the opcode in IR into the microaddress where that instruction's microprogram begins in CM, and loads it into MPC.

- This is the microprogrammed counterpart of Week 6's opcode decoder (D_i lines) — same job ("which instruction is this?"), different output: an *address* into memory, not a one-hot gate-enable line.
- After the mapped starting address loads, MPC advances using the **next-address field** carried in each control word (Act 1) — normally the next row, until END triggers a new fetch and a fresh mapping.

Two Sources Feed MPC



Flags let a control word's next-address field *branch* rather than simply advance — how a conditional instruction (e.g. a compare-and-jump) selects between two successors [2].

Worked Trace Revisited: Where MPC Points

Step	MPC value	CM row read	RTL
Fetch	mapped $\rightarrow 0$	row 0	$Y \leftarrow [R2]$
$+1$	1	row 1	$Z \leftarrow [Y] + [R3]$
$+1$	2	row 2 (END)	$R1 \leftarrow [Z]$, then remap on next fetch

This is the piece that plays SC's role: mapping plus the next-address field is what turns "read row 0, then 1, then 2" from something we listed by hand into something the circuit does on its own.

Socratic Check: What If Mapping Were Missing?

Question

Suppose the mapping unit were removed and MPC always reset to 0 after END. What breaks?

- Every instruction would execute CM row 0's microprogram — whichever instruction happens to occupy that address — regardless of what is actually in IR.
- Exactly like Week 6's missing SC_{clear}, this is not optional bookkeeping: mapping is what lets a shared CM serve *every* instruction instead of just one.

But different instructions need different micro-operations at each step — how wide is one control word?

The Missing Piece: How Wide Is a Control Word?

Socratic question

Our worked microprogram used about eight distinct signals ($R2_{\text{out}}$, $R3_{\text{out}}$, Y_{in}, ALU=add, Z_{in}, Z_{out}, $R1_{\text{in}}$, END). A real ISA has dozens to hundreds. Does every control word need one bit *per signal*, even though only a handful are ever active at once?

This is a genuine width-vs.-simplicity trade-off, and it has a name: **horizontal** vs. **vertical** microinstruction encoding.

Definition: Horizontal Microinstruction

Horizontal encoding

One bit per control signal. Bit k of the control word *is* X_{in} (or $R2_{\text{out}}$, or END) directly — reading the word out of CM *is* asserting the signals, with no further logic in between.

- Pro: no decode delay — signals are valid the instant CM is read, same cycle.
- Con: the word is as wide as the number of signals in the whole machine, and most bits are 0 in any given row.

Definition: Vertical Microinstruction

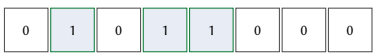
Vertical encoding

Signals that are mutually exclusive (e.g. “which register drives the bus this cycle” has exactly one answer) are grouped into a small *encoded field*. A **decoder** between CM and the datapath expands each field back into the actual one-hot signal.

- Pro: far narrower control word — n mutually-exclusive signals need only $\lceil \log_2 n \rceil$ bits, not n.
- Con: the decoder is on the critical path — signals are valid one decode delay *after* CM is read, not the same instant.

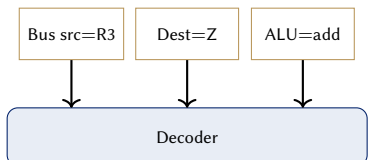
Side by Side: Encoding Our Worked Example

Horizontal (row 1: T_2)



8 bits, one per signal — read directly, no decoder.

Vertical (row 1: T_2)



3 fields ($\approx$6 bits) — needs a decode step before signals are valid.

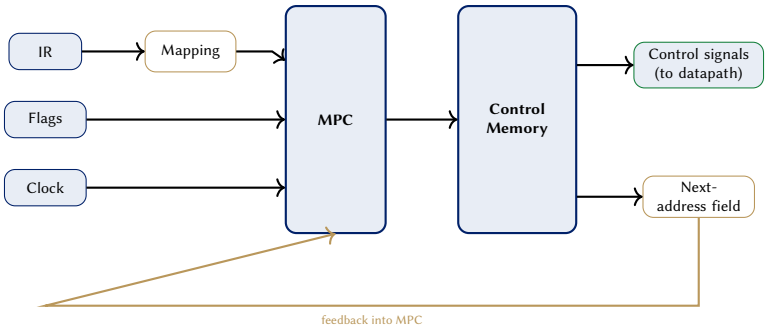
Horizontal vs. Vertical: Trade-offs

Width vs. speed

- Horizontal: no decode delay, but CM width grows linearly with the number of signals in the machine — at real-ISA scale, 100–400 bits per row is typical.
- Vertical: CM width shrinks to $\sim \log_2$(signals per group), summed across groups — but every control word pays a decoder's worth of extra delay before the datapath sees valid signals.
- Our toy example (8 bits $\rightarrow \approx$6 bits) barely shows the gain — the saving compounds with instruction-set size, which is exactly where hardwired control's design cost was exploding.

Put the two new hardware pieces together — what does the full control unit look like now?

Full Microprogrammed Control Unit Block Diagram



[2, 1]

Socratic Check: Adding a New Instruction

Question

Recall Week 6: adding MUL meant a new opcode-decoder line *and* rewiring an AND gate into an existing OR gate on the chip. What does adding MUL cost here?

- Write MUL's control words into *unused* rows of CM — no new gates.
- Extend the mapping unit with one more (opcode → starting microaddress) entry — typically itself a small memory/table, not hand-derived logic.
- The chip's gates do not change at all; only the contents of a memory do.

Microprogrammed Control: Trade-offs

Flexibility vs. speed

- Pro: new or modified instructions are a memory rewrite, not a redesign — even a bug fix in an already-fabricated CPU can ship as a microcode patch (real x86 chips do exactly this).
- Pro: the state table *is* the design — no hand-derived Boolean equations to re-verify.
- Con: every control step costs a memory read (and a decode step, if vertical) — strictly slower per cycle than hardwired's pure combinational logic.
- Con: CM itself is extra hardware that hardwired control does not need at all.

Hardwired vs. Microprogrammed: Head to Head

	Hardwired (Week 6)	**Microprogrammed (this week)**
Speed per step	Fastest — pure gates	Slower — memory read (+ decode)
Adding an instruction	New/re-wired gates	New CM rows
Design method	Derive Boolean equations	Write down control words
Typical fit	Small, fixed, speed-critical ISAs (RISC)	Large, evolving ISAs (classical CISC)

Same control problem, two implementations — the choice trades raw speed against design and maintenance cost, exactly the pattern Weeks 5–6 already established for datapaths.

The Three Threads Meet

Every design choice moves cost somewhere else on the chip

- **Bus organization (Week 5):** single-bus needs 3 control steps for our trace; three-bus needs 1 — fewer steps, more wiring.
- **Control-unit style (Weeks 6–7):** hardwired is fastest per step but rigid; microprogrammed is slower per step but just a memory rewrite to change.
- **Encoding width (this week):** horizontal avoids decode delay but costs CM width; vertical shrinks CM but adds a decoder to the critical path.

None of these is free — every one trades cycle time, design effort, or raw hardware for one of the other two.

Bridge to Week 8

From controlling the chip to controlling the world

Weeks 6–7 answered "how does the CPU decide what *itself* does, every cycle?" Week 8 asks the next question: how does the CPU decide what *outside* devices — keyboards, disks, printers — do, and when?

- I/O devices are far slower than the CPU — naively waiting for one wastes thousands of cycles.
- Interrupts, polling, DMA, and I/O processors are four different answers to "who watches the device, and what happens when it's ready?"

Summary

- **Control word (microinstruction)** = one stored row of control signals plus next-address information; a **microprogram** is the ordered sequence implementing one instruction.
- **Control memory (CM)** + **microprogram counter (MPC)** replace the control matrix and SC: MPC addresses CM, whose output is the control word.
- **Address sequencing**: a **mapping** unit loads MPC with an instruction's starting microaddress; each control word's next-address field advances or branches MPC thereafter.
- **Horizontal** microinstructions (one bit per signal, no decode delay) vs. **vertical** (encoded fields, decoder required, narrower words) trade width against speed.
- **Hardwired vs. microprogrammed**: same control problem, opposite trade-offs — speed and rigidity vs. a memory-read delay and easy modification.

References


V. Carl Hamacher, Zvonko G. Vranesic, and Safwat G. Zaky.
Computer Organization.
McGraw-Hill, 5th edition, 2002.


M. Morris Mano.
Computer System Architecture.
Prentice-Hall, 3rd edition, 1993.


William Stallings.
Computer Organization and Architecture: Designing for Performance.
Pearson, 10th edition, 2015.